

MyelinFatigueNet: A Dual-Stream Recurrent Architecture with Bidirectional Gating and Neural Fatigue Regularization for Continuous Sequence Modeling

Anonymous Authors

Preprint. Correspondence to: mfn-paper@example.org

ABSTRACT

Classical recurrent architectures maintain a single hidden state that must simultaneously encode fast-varying surface patterns and slow-drifting contextual representations, creating a representational bottleneck. We introduce **MyelinFatigueNet (MFN)**, a recurrent architecture that decomposes internal dynamics into two parallel, bidirectionally coupled streams: an *expressive stream* $\mathbf{h}_\Lambda$ tracking high-frequency input variation, and a *conceptual stream* $\mathbf{h}_\Psi$ integrating semantic context over long horizons. Timescale separation is enforced structurally through independent input-conditioned adaptive decay modules. Two directional confidence gates — $g_{\Psi \rightarrow \Lambda}$ and $g_{\Lambda \rightarrow \Psi}$ — mediate cross-stream communication in both directions, each learning a distinct gating policy. A per-stream neural fatigue accumulator ϕ_s , grounded in synaptic short-term depression, prevents representational fixation by suppressing over-active neurons and forcing activation diversity across time. We derive the full parameter budget ($22H^2 + (2d + 2d_{\text{out}} + 17)H + d_{\text{out}}$), verify the architecture against a corrected reference implementation, and design three discriminative evaluation protocols: multi-delay copy, character-level language modeling on Penn Treebank, and a dual-context tracking task that directly tests simultaneous parallel memory maintenance.

Keywords: recurrent neural networks · multi-timescale learning · neural fatigue · bidirectional gating · adaptive decay · streaming inference · sequence modeling

1. INTRODUCTION

The dominant paradigm for sequence modeling is the Transformer [1], which achieves state-of-the-art performance by attending globally over an input context via scaled dot-product attention. Despite this success, Transformers are feedforward at inference time: each token generation step is conditioned only on a static, monotonically growing key-value cache. No dynamic signal propagates between generation steps.

Recurrent architectures — vanilla RNNs, LSTMs [2], and GRUs [3] — maintain a dynamic hidden state updated at every timestep, making them natural candidates for streaming and real-time settings. However, they suffer from two structural limitations. First, their single hidden state must simultaneously represent both fast-changing and slow-drifting signal components, a representational bottleneck. Second, they provide no mechanism for preventing representational fixation: once a neuron subpopulation captures a strong pattern, nothing inhibits it from persisting indefinitely.

Recent work on state-space models — S4 [6], Mamba [5] — revisits recurrence with structured state transitions enabling selective memory retention. More recent recurrent revivals — xLSTM [14], Griffin [15], RWKV [16], RetNet [17] — improve expressivity and throughput. None of these architectures model *synaptic fatigue* or decompose their state into interpretable timescale streams with bidirectional coupling.

We propose **MyelinFatigueNet (MFN)**, which addresses all three limitations through four complementary mechanisms:

1. **Dual streams $\mathbf{h}_\Lambda$** (expressive) and $\mathbf{h}_\Psi$ (conceptual), operating at structurally distinct timescales within a single recurrent layer.
2. **Independent adaptive decay** via two separate input-conditioned MLPs,

allowing each stream to adapt its forgetting rate independently to the current input.

3. **Bidirectional confidence gates** $g_{\Psi \rightarrow \Lambda}$ and $g_{\Lambda \rightarrow \Psi}$ with reversed argument order, each learning a distinct directional gating policy.
4. **Neural fatigue accumulators** $\phi_s \in [0, 1]^H$ per stream, grounded in synaptic short-term depression, which suppress over-active neurons and force representational diversity.

Together these mechanisms produce a continuously evolving, self-regulating internal state that better reflects the parallel structure of biological cognition. The architecture runs in $O(H^2)$ per step with constant memory, making it suitable for continuous or real-time generation.

2. RELATED WORK

2.1 Transformer Models

GPT-style autoregressive Transformers [1,7] generate tokens by attending over all previous positions via causal self-attention. The key-value cache stores past representations but is never rewritten — it grows monotonically. Recurrent Transformers such as Transformer-XL [8] reintroduce a segment memory, but it remains a fixed snapshot. MFN differs by maintaining a dynamic, gated, fatigue-modulated hidden state updated at every timestep with constant memory cost.

2.2 LSTM and GRU

The LSTM [2] introduces input, forget, and output gates to control information flow into a cell state, enabling gradient propagation over longer sequences [4]. The GRU [3] simplifies this to two gates with comparable empirical performance. Both maintain a single hidden state that must represent all temporal horizons simultaneously. MFN decomposes this into two streams with structurally enforced timescale separation, adds bidirectional cross-stream gating, and introduces the

fatigue accumulator as a third regulatory layer.

2.3 Recent Recurrent Revivals

xLSTM [14] introduces exponential gating and multi-head memory matrices, achieving competitive language modeling. Griffin [15] (Google DeepMind) combines linear recurrence with local attention. RWKV [16] reformulates attention as a linear recurrence enabling linear-cost inference. RetNet [17] introduces explicit multi-scale retention heads with fixed geometric decay rates. None of these architectures model synaptic fatigue or implement bidirectional coupling between explicitly separated timescale streams.

2.4 Hierarchical and Clock-Driven RNNs

Hierarchical Multiscale RNNs [9] organise recurrent layers into a temporal hierarchy where lower layers update more frequently via learned boundary detectors. The Clockwork RNN [10] assigns fixed update periods to neuron groups. MFN shares the timescale decomposition intuition but implements it within a single layer via independent adaptive decay MLPs rather than stacking layers or assigning clock rates, and couples the streams bidirectionally through separate learned gates.

2.5 State-Space Models

S4 [6] parameterises recurrence via HiPPO-initialised state matrices. Mamba [5] adds input-dependent transition matrices (selective SSMs) and achieves $O(H)$ per-step cost. MFN is less parameter-efficient but offers explicit stream interpretability: $\mathbf{h}_\Lambda$ and $\mathbf{h}_\Psi$ can be probed independently. Mamba's Δ (discretisation step) is functionally analogous to MFN's α , but applies to a single structured state without timescale decomposition or fatigue.

2.6 Predictive Coding and Synaptic Fatigue

Predictive coding [11] models cortical processing as a hierarchy of prediction-error signals. PredNet [12] implements this for video prediction. MFN borrows the

bidirectional feedback intuition but replaces prediction-error with a learned confidence gate applicable to arbitrary tasks. The neural fatigue mechanism is grounded in synaptic short-term depression [13]: sustained presynaptic firing depletes vesicle pools, transiently reducing postsynaptic response amplitude — precisely the dynamic modeled by ϕ .

3. ARCHITECTURE

3.1 Overview and Notation

Let H be the hidden dimension, d the input dimension, and d_{out} the output dimension. MFN maintains two hidden states $\mathbf{h}_\Lambda, \mathbf{h}_\Psi \in \mathbb{R}^H$ and two fatigue accumulators $\phi_\Lambda, \phi_\Psi \in [0, 1]^H$. At each timestep t , the model receives $\mathbf{x}_t \in \mathbb{R}^d$, updates both streams sequentially, refreshes the fatigue state, and produces output $\mathbf{y}_t \in \mathbb{R}^{d_{\text{out}}}$. All learned per-neuron scalars are stored as unconstrained parameters and passed through $\sigma(\cdot)$ at use time to enforce the required range constraints.

The symbol

MathJax maximum macro substitution count exceeded; is here ;
denotes elementwise multiplication. We write $[\mathbf{a}; \mathbf{b}]$ for concatenation along the feature axis.

3.2 Input Projection

$$\tilde{\mathbf{x}}_t = W_{\text{in}} \mathbf{x}_t + \mathbf{b}_{\text{in}}, \quad W_{\text{in}} \in \mathbb{R}^{H \times d}$$

The raw input is projected to the hidden dimension H shared by both streams. A single shared projection is used because both streams receive identical input — their differing responses arise from distinct decay rates and gate policies, not from distinct feature extractors.

3.3 Neural Fatigue Accumulator

Synaptic short-term depression [13] arises from the transient depletion of readily releasable neurotransmitter vesicles during sustained activation. We model this with a per-stream, per-neuron fatigue state $\phi_{s,t} \in [0, 1]^H$ for $s \in \{\Lambda, \Psi\}$:

MathJax maximum macro substitution count exceeded; is

where $\gamma_s = \sigma(\mathbf{w}_{\gamma,s}) \in (0, 1)^H$ is a learned per-neuron recovery rate. A neuron with high $\gamma_{s,i}$ recovers slowly (persistent fatigue); one with low $\gamma_{s,i}$ recovers quickly (transient fatigue). The fatigue update is computed *after* the GRU step, so $\phi_{s,t}$ reflects the activation just computed and influences only the *next* timestep.

The **effective (fatigue-modulated) hidden state** used in all downstream computations is:

MathJax maximum macro substitution count exceeded; is

When $\phi_{s,t,i} \rightarrow 1$, neuron i is saturated and its contribution to outputs, gates, and cross-stream feedback is suppressed toward zero. This forces the network to recruit less-fatigued neurons, creating a natural rotation of representational load — implicit attention without the quadratic cost of self-attention.

3.4 Bidirectional Confidence Gates

Two independent gates mediate cross-stream communication, each computed from the joint fatigue-modulated state:

$$g_{\Psi \rightarrow \Lambda, t} = \sigma\left(W_{g1} [\tilde{\mathbf{h}}_{\Psi, t-1}; \tilde{\mathbf{h}}_{\Lambda, t-1}] + \mathbf{b}_{g1}\right) \quad (3)$$

$$g_{\Lambda \rightarrow \Psi, t} = \sigma\left(W_{g2} [\tilde{\mathbf{h}}_{\Lambda, t-1}; \tilde{\mathbf{h}}_{\Psi, t-1}] + \mathbf{b}_{g2}\right) \quad (4)$$

where $W_{g1}, W_{g2} \in \mathbb{R}^{H \times 2H}$ and $g_{\cdot, t} \in (0, 1)^H$. The argument order is *reversed* between the two gates: $g_{\Psi \rightarrow \Lambda}$ is computed with the conceptual state placed first,

while $g_{\Lambda \rightarrow \Psi}$ places the expressive state first. This asymmetry gives each gate a distinct projection of the joint state, enabling independent directional policies to emerge during training without increasing the parameter count.

Because the gates are computed from fatigue-modulated states $\tilde{\mathbf{h}}$, a stream that is currently fatigued automatically exerts weaker cross-stream influence, preventing stale representations from corrupting the other stream.

3.5 Independent Adaptive Decay

A single shared decay signal cannot allow the two streams to adapt their forgetting rates independently in response to the same input. MFN uses two separate two-layer MLPs:

$$\alpha_{\Lambda,t} = \sigma\left(W_{\alpha,2}^{\Lambda} \text{ReLU}\left(W_{\alpha,1}^{\Lambda} \mathbf{x}_t + \mathbf{b}_{\alpha,1}^{\Lambda}\right) + \mathbf{b}_{\alpha,2}^{\Lambda}\right) \in (0, 1)^H \quad (5)$$

$$\alpha_{\Psi,t} = \left[\sigma\left(W_{\alpha,2}^{\Psi} \text{ReLU}\left(W_{\alpha,1}^{\Psi} \mathbf{x}_t + \mathbf{b}_{\alpha,1}^{\Psi}\right) + \mathbf{b}_{\alpha,2}^{\Psi}\right)\right]^{\beta} \in (0, 1)^H \quad (6)$$

with $\beta = 0.2$. Because $x \in (0, 1) \Rightarrow x^{\beta} > x$ for $\beta < 1$, the conceptual stream's effective decay values are structurally closer to 1 (slower forgetting) for any MLP output, enforcing timescale separation at the parameter level rather than through initialisation or regularisation. The two decay MLPs are parameterised independently, so the expressive stream can accelerate its forgetting in response to an abrupt input change without forcing the same on the conceptual stream.

3.6 Feedback Terms

Within-stream feedback is gated by the confidence gate originating in the *other* stream:

MathJax maximum macro substitution count exceeded; is

MathJax maximum macro substitution count exceeded; is

where $W_{fb,\Lambda}, W_{fb,\Psi} \in \mathbb{R}^{H \times H}$. The inner stream gates expression ($\Psi \rightarrow \Lambda$): the expressive stream's self-feedback is modulated by how coherently the conceptual stream represents the current context. Conversely, the expressive stream gates conceptual feedback ($\Lambda \rightarrow \Psi$): the conceptual stream updates more strongly when the expressive stream carries new semantic content. Fatigue enters feedback through the $\tilde{\mathbf{h}}$ terms, so a fatigued stream automatically sends weaker self-reinforcement.

3.7 Sequential Stream Updates

The two streams are updated *sequentially* within each timestep rather than in parallel. This ordering is deliberate:

$$\mathbf{h}_{\Lambda,t} = \text{GRUCell}_{\Lambda}(\tilde{\mathbf{x}}_t + \mathbf{f}_{\Lambda,t} + W_{\Psi \rightarrow \Lambda} \tilde{\mathbf{h}}_{\Psi,t-1}, \mathbf{h}_{\Lambda,t-1}) \quad (9)$$

A tentative effective state is then formed for use as cross-stream input:

MathJax maximum macro substitution count exceeded; is

Note that $\tilde{\mathbf{h}}_{\Lambda,t}^*$ is modulated by the *previous* fatigue $\phi_{\Lambda,t-1}$ because the fatigue update for timestep t (Eq. 1) has not yet been applied. Then:

$$\mathbf{h}_{\Psi,t} = \text{GRUCell}_{\Psi}(\tilde{\mathbf{x}}_t + \mathbf{f}_{\Psi,t} + W_{\Lambda \rightarrow \Psi} \tilde{\mathbf{h}}_{\Lambda,t}^*, \mathbf{h}_{\Psi,t-1}) \quad (11)$$

where $W_{\Psi \rightarrow \Lambda}, W_{\Lambda \rightarrow \Psi} \in \mathbb{R}^{H \times H}$ (no bias) are learned cross-stream projection matrices. The expressive stream Λ is updated first, incorporating the *previous* conceptual state $\tilde{\mathbf{h}}_{\Psi,t-1}$. The conceptual stream Ψ is then updated using the *freshly computed* $\tilde{\mathbf{h}}_{\Lambda,t}^*$. This models the cognitive asymmetry where background reasoning integrates what has just been expressed before updating, while expression is primarily driven by the previous thought state.

3.8 Fatigue Update

After both GRU updates, the fatigue accumulators are refreshed (Eq. 1):

MathJax maximum macro substitution count exceeded; is

MathJax maximum macro substitution count exceeded; is

Placing this step *after* the GRU updates ensures that $\phi_{s,t}$ reflects the activations at time t and will suppress only the *next* timestep's computation, not the current one. The final effective states for the readout incorporate the freshly updated fatigue:

MathJax maximum macro substitution count exceeded; is

3.9 Readout

$$\mathbf{y}_t = W_{\text{out}} [\tilde{\mathbf{h}}_{\Lambda,t}; \tilde{\mathbf{h}}_{\Psi,t}] + \mathbf{b}_{\text{out}}, \quad W_{\text{out}} \in \mathbb{R}^{d_{\text{out}} \times 2H} \quad (15)$$

The readout uses exclusively fatigue-modulated states. Neurons that have been over-active cannot dominate the output regardless of their raw magnitude in $\mathbf{h}$. This contrasts with standard GRU/LSTM readouts, which apply no representational ceiling.

4. PARAMETER BUDGET

We provide a complete derivation of the parameter count. For GRU cells, we adopt the convention that each gate has a single combined bias vector of dimension H (rather than the two separate bias vectors used in some PyTorch defaults). This is a common simplification in architectural analysis that can be reverted at negligible cost.

--	--	--	--

Component	Parameters	H^2 coefficient	Linear
Input projection $W_{\text{in}}, \mathbf{b}_{\text{in}}$	$dH + H$	0	$dH +$
Feedback projections $W_{fb,\Lambda},$ $W_{fb,\Psi} (\times 2)$	$2(H^2 + H)$	2	$2H$
Cross-stream projections $W_{\Psi \rightarrow \Lambda},$ $W_{\Lambda \rightarrow \Psi} (\times 2,$ no bias)	$2H^2$	2	—
Confidence gates $W_{g1},$ $W_{g2} (\times 2,$ input dim $= 2H)$	$2(2H^2 + H)$	4	$2H$
Adaptive decay MLPs ($\times 2$ independent, $d \rightarrow H \rightarrow H$)	$2(dH + H + H^2 + H) = 2dH + 2H^2 + 4H$	2	$2dH +$
GRU cells ($\times 2$, input dim $= H$, 3 gates each)	$2 \times 3(2H^2 + H) = 12H^2 + 6H$	12	$6H$
Fatigue recovery rates $\mathbf{w}_{\gamma,\Lambda},$ $\mathbf{w}_{\gamma,\Psi}$	$2H$	0	$2H$

Readout $W_{\text{out}}, \mathbf{b}_{\text{out}}$	$2Hd_{\text{out}} + d_{\text{out}}$	0	$2d_{\text{out}}I$
Total	$22H^2 + (3d + 2d_{\text{out}} + 15)H + d_{\text{out}}$		

Table 1: Parameter breakdown of MFN. Each row is independently verifiable. The cross-stream projections (row 3) are a component absent from some prior related formulations, accounting for $2H^2$ of the total.

Numerical verification. For $H = 64, d = 10, d_{\text{out}} = 10$:

$$22 \times 64^2 + (30 + 20 + 15) \times 64 + 10 = 90,112 + 4,160 + 10 = \mathbf{94,282} \text{ parameters}$$

The inclusion of $2H^2$ cross-stream projection parameters — absent from comparable single-stream architectures — represents a deliberate design choice enabling bidirectional latent communication. For a variant without explicit cross-stream projections (where cross-stream information enters only through the gate and feedback), the total reduces to $20H^2 + (3d + 2d_{\text{out}} + 15)H + d_{\text{out}} \approx 82,090$ at these dimensions.

The dominant term is the two GRU cells, contributing $12H^2$ ($\approx 65\%$ of the quadratic cost at large H). Structured or low-rank factorisation of the GRU weight matrices (e.g., Monarch factorisation [18]) would reduce this without changing the architectural logic.

5. COGNITIVE AND BIOLOGICAL GROUNDING

5.1 Stream Roles

The expressive stream $\mathbf{h}_\Lambda$ models surface-level cognitive output: rapid, high-frequency tracking of input that drives moment-to-moment responses. In language generation this corresponds to the articulatory or phonological loop — the "inner voice" shaping each word choice. Its fast adaptive decay keeps it responsive to recent input at the cost of long-range retention.

The conceptual stream $\mathbf{h}_\Psi$ models background reasoning: slower accumulation of semantic and contextual meaning that gives expression its coherence over longer horizons. Its structurally constrained decay ($\beta = 0.2$) keeps effective decay values

close to 1, allowing it to maintain a train of thought across many expressive steps. Crucially, $\mathbf{h}_\Psi$ is never directly output — it influences $\mathbf{y}_t$ only through $\tilde{\mathbf{h}}_{\Psi,t}$ in the readout and through $g_{\Psi \rightarrow \Lambda}$ on the expressive stream.

5.2 Neural Fatigue as Representational Regulation

Synaptic fatigue in biological networks arises from the depletion of readily releasable neurotransmitter vesicles during sustained activation [13]. Recovery occurs at a timescale governed by vesicle replenishment rates — from tens of milliseconds to several seconds depending on synapse type. MFN's fatigue accumulator directly models this: the exponential decay of ϕ corresponds to recovery, and $\gamma_{s,i}$ is the learned neuron-specific replenishment timescale.

Functionally, fatigue prevents fixation. When one set of neurons has been driving the output for several steps, their fatigue rises, $\tilde{\mathbf{h}}$ falls, and the gate and readout naturally shift weight toward neurons that have been recovering. This creates implicit attention-like rotation over the hidden population without any explicit attention mechanism.

5.3 Bidirectional Gates as Directional Coherence Policies

$g_{\Psi \rightarrow \Lambda}$ answers: *"How much should current background reasoning guide expression?"* — high when the conceptual stream has a coherent representation, low when it is uncertain or in flux. $g_{\Lambda \rightarrow \Psi}$ answers: *"How much should what is being expressed update the underlying thought?"* — high when the expressive output carries novel semantic content, low when it is routine or predictable. Learning these policies from data allows the model to discover when to let thoughts guide words and when to let words update thoughts.

6. COMPARISON WITH EXISTING ARCHITECTURES

Property	Transformer	GRU / LSTM	Mamba	xLSTM	Griffin	MFN (ours)

Dynamic hidden state	✗	✓	✓	✓	✓	✓
Multi-timescale	✗	✗	✗	✗	Partial	✓ (2 streams)
Bidirectional coupling	✗	✗	✗	✗	✗	✓ (2 gates)
Neural fatigue	✗	✗	✗	✗	✗	✓ (ϕ per stream)
Input-cond. decay	—	✗	✓ (Δ)	Partial	✓	✓ (independent MLPs)
Streaming inference	Approx.	✓	✓	✓	✓	✓
Cost per step	$O(n^2)$	$O(H^2)$	$O(H)$	$O(H^2)$	$O(H)$	$O(H^2)$
Interpretable timescales	✗	✗	✗	✗	✗	✓
Fatigue-based diversity	✗	✗	✗	✗	✗	✓

Table 2: Qualitative comparison of MFN against dominant sequence modeling architectures. MFN is the only architecture with all of: bidirectional coupling, neural fatigue, and independent multi-timescale adaptive decay.

7. REFERENCE ALGORITHM AND IMPLEMENTATION

7.1 Forward Pass Pseudo-Code

ALGORITHM 1 — MFN FORWARD PASS (ONE TIMESTEP $\mathcal{T}$)

1. **Input:** $\mathbf{x}_t \in \mathbb{R}^d$; states $(\mathbf{h}_\Lambda, \mathbf{h}_\Psi, \phi_\Lambda, \phi_\Psi)$ from $t - 1$
2. **Input projection:** $\tilde{\mathbf{x}}_t \leftarrow W_{\text{in}}\mathbf{x}_t + \mathbf{b}_{\text{in}}$
3. **Recovery rates:** $\gamma_\Lambda \leftarrow \sigma(\mathbf{w}_{\gamma,\Lambda}); \gamma_\Psi \leftarrow \sigma(\mathbf{w}_{\gamma,\Psi})$

4. Effective states (using prior fatigue):

MathJax maximum macro substitution count exceeded; is here a re
MathJax maximum macro substitution count exceeded; is here a re

5. Adaptive decay:

$$\alpha_{\Lambda} \leftarrow \sigma(\text{MLP}_{\Lambda}(\mathbf{x}_t))$$

$$\alpha_{\Psi} \leftarrow \sigma(\text{MLP}_{\Psi}(\mathbf{x}_t))^{\beta}$$

6. Bidirectional gates:

$$g_{\Psi \rightarrow \Lambda} \leftarrow \sigma(W_{g1}[\tilde{\mathbf{h}}_{\Psi}; \tilde{\mathbf{h}}_{\Lambda}] + \mathbf{b}_{g1})$$

$$g_{\Lambda \rightarrow \Psi} \leftarrow \sigma(W_{g2}[\tilde{\mathbf{h}}_{\Lambda}; \tilde{\mathbf{h}}_{\Psi}] + \mathbf{b}_{g2})$$

7. Feedback terms:

MathJax maximum macro substitution count exceeded; is here a re
MathJax maximum macro substitution count exceeded; is here a re

8. Update Λ (expressive, first):

$$\mathbf{h}_{\Lambda} \leftarrow \text{GRUCell}_{\Lambda}(\tilde{\mathbf{x}}_t + \mathbf{f}_{\Lambda} + W_{\Psi \rightarrow \Lambda} \tilde{\mathbf{h}}_{\Psi}, \mathbf{h}_{\Lambda})$$

9. Tentative effective Λ (for Ψ input, uses prior ϕ_{Λ}):

MathJax maximum macro substitution count exceeded; is here a re

10. Update Ψ (conceptual, second):

$$\mathbf{h}_{\Psi} \leftarrow \text{GRUCell}_{\Psi}(\tilde{\mathbf{x}}_t + \mathbf{f}_{\Psi} + W_{\Lambda \rightarrow \Psi} \tilde{\mathbf{h}}_{\Lambda}^*, \mathbf{h}_{\Psi})$$

11. Fatigue update (post-GRU):

MathJax maximum macro substitution count exceeded; is here a re
MathJax maximum macro substitution count exceeded; is here a re

12. Final effective states and readout:

MathJax maximum macro substitution count exceeded; is here a re
MathJax maximum macro substitution count exceeded; is here a re

$$\mathbf{y}_t \leftarrow W_{\text{out}}[\tilde{\mathbf{h}}_{\Lambda}; \tilde{\mathbf{h}}_{\Psi}] + \mathbf{b}_{\text{out}}$$

13. Return: $\mathbf{y}_t$ and updated $(\mathbf{h}_{\Lambda}, \mathbf{h}_{\Psi}, \phi_{\Lambda}, \phi_{\Psi})$

Initialization. $\mathbf{h}_{\Lambda}^{(0)} = \mathbf{h}_{\Psi}^{(0)} = \mathbf{0}$; $\phi_{\Lambda}^{(0)} = \phi_{\Psi}^{(0)} = \mathbf{0}$ (no prior fatigue). $\mathbf{w}_{\gamma,s}$ initialized to 0 (so $\sigma(\mathbf{w}_{\gamma}) = 0.5$ initially — moderate recovery speed). Decay MLP weights use standard Xavier initialization.

7.2 PyTorch Reference Implementation

```

import torch
import torch.nn as nn

class MyelinFatigueNet(nn.Module):
    """Dual-stream recurrent architecture with bidirectional gating
    and neural fatigue regularization.

    Args:
        input_size (int): Input dimension d.
        hidden_size (int): Per-stream hidden dimension H.
        output_size (int): Output dimension d_out.
        beta (float): Slow-stream decay exponent (default 0.2).
    """
    def __init__(self, input_size: int, hidden_size: int,
                  output_size: int, beta: float = 0.2):
        super().__init__()
        H, d = hidden_size, input_size
        self.beta = beta

        # Shared input projection
        self.input_proj = nn.Linear(d, H)

        # Independent adaptive decay MLPs ( $d \rightarrow H \rightarrow H$ )
        self.decay_lam = nn.Sequential(
            nn.Linear(d, H), nn.ReLU(), nn.Linear(H, H), nn.Sigmoid())
        self.decay_psi = nn.Sequential(
            nn.Linear(d, H), nn.ReLU(), nn.Linear(H, H), nn.Sigmoid())

        # Within-stream feedback projections
        self.fb_lam = nn.Linear(H, H)
        self.fb_psi = nn.Linear(H, H)

        # Cross-stream projections (no bias, to avoid ambiguity
        # with the gate's additive bias)
        self.W_psi2lam = nn.Linear(H, H, bias=False)
        self.W_lam2psi = nn.Linear(H, H, bias=False)

        # Bidirectional confidence gates (input dim = 2H)
        self.gate_psi2lam = nn.Sequential(
            nn.Linear(2 * H, H), nn.Sigmoid()) # Psi → Lambda
        self.gate_lam2psi = nn.Sequential(
            nn.Linear(2 * H, H), nn.Sigmoid()) # Lambda → Psi

```

```

# GRU update cells
self.gru_lam = nn.GRUCell(H, H)
self.gru_psi = nn.GRUCell(H, H)

# Per-neuron fatigue recovery rates (stored as raw logits;
# sigmoid applied at runtime to enforce (0, 1) range)
self.w_gamma_lam = nn.Parameter(torch.zeros(H))
self.w_gamma_psi = nn.Parameter(torch.zeros(H))

# Readout (concatenated streams → output)
self.readout = nn.Linear(2 * H, output_size)

def forward(self, x, h_lam, h_psi, phi_lam, phi_psi):
    """Single-step forward pass.

    Args:
        x          : (B, d)    current input
        h_lam       : (B, H)    expressive hidden state
        h_psi       : (B, H)    conceptual hidden state
        phi_lam     : (B, H)    expressive fatigue in [0, 1]
        phi_psi     : (B, H)    conceptual fatigue in [0, 1]

    Returns:
        h_lam, h_psi, phi_lam, phi_psi  -- updated states
        y                                -- (B, d_out) output
    """
    # --- Recovery rates (per neuron, enforced in (0,1)) ---
    gamma_lam = torch.sigmoid(self.w_gamma_lam)  # (H,)
    gamma_psi = torch.sigmoid(self.w_gamma_psi)  # (H,)

    # --- Fatigue-modulated effective states (prior fatigue) ---
    h_lam_eff = h_lam * (1.0 - phi_lam)          # (B, H)
    h_psi_eff = h_psi * (1.0 - phi_psi)          # (B, H)

    # --- Input projection ---
    xp = self.input_proj(x)                      # (B, H)

    # --- Adaptive decay ---
    alpha_lam = self.decay_lam(x)                 # (B, H) in
    alpha_psi = self.decay_psi(x) ** self.beta    # (B, H) sl

```

```

# --- Bidirectional confidence gates ---
# Argument order is REVERSED between the two gates
joint_p2l = torch.cat([h_psi_eff, h_lam_eff], dim=-1) # (B, 2H)
joint_l2p = torch.cat([h_lam_eff, h_psi_eff], dim=-1) # (B, 2H)
g_p2l = self.gate_psi2lam(joint_p2l) # (B, H)
g_l2p = self.gate_lam2psi(joint_l2p) # (B, H)

# --- Within-stream feedback (gated by the OTHER stream) ---
fb_lam = self.fb_lam(h_lam_eff) * alpha_lam * g_p2l # (B, H)
fb_psi = self.fb_psi(h_psi_eff) * alpha_psi * g_l2p # (B, H)

# --- Stream updates: Lambda FIRST, then Psi uses fresh h_lam ---
h_lam = self.gru_lam(
    xp + fb_lam + self.W_psi2lam(h_psi_eff), h_lam) # (B, H)

# Tentative effective Lambda for Psi input (uses PRIOR phi_lam)
h_lam_eff_star = h_lam * (1.0 - phi_lam) # (B, H)

h_psi = self.gru_psi(
    xp + fb_psi + self.W_lam2psi(h_lam_eff_star), h_psi) # (B, H)

# --- Fatigue update (post-GRU, affects next step only) ---
phi_lam = gamma_lam * phi_lam + (1.0 - gamma_lam) * h_lam_eff_star
phi_psi = gamma_psi * phi_psi + (1.0 - gamma_psi) * h_psi_eff_star

# --- Final effective states and readout ---
h_lam_out = h_lam * (1.0 - phi_lam)
h_psi_out = h_psi * (1.0 - phi_psi)
y = self.readout(torch.cat([h_lam_out, h_psi_out], dim=-1))

return h_lam, h_psi, phi_lam, phi_psi, y

```

```

@staticmethod

```

```

def init_state(batch_size: int, hidden_size: int, device='cpu'):
    """Return zero-initialised (h_lam, h_psi, phi_lam, phi_psi)
    zeros = lambda: torch.zeros(batch_size, hidden_size, device=device)
    return zeros(), zeros(), zeros(), zeros()

```

```

# — Sequence-level loop —————
def run_sequence(model, x_seq, batch_size, hidden_size, device='cpu'):
    """

```

```

x_seq : (T, B, d)
returns y_seq : (T, B, d_out)
"""
states = MyelinFatigueNet.init_state(batch_size, hidden_size, c
h_lam, h_psi, phi_lam, phi_psi = states
outs = []
for x_t in x_seq:                                # iterate over T
    h_lam, h_psi, phi_lam, phi_psi, y_t = \
        model(x_t, h_lam, h_psi, phi_lam, phi_psi)
    outs.append(y_t)
return torch.stack(outs)                          # (T, B, d_out)

```

8. EXPERIMENTAL PROTOCOL

8.1 Baselines and Parameter Matching

All baselines are parameter-matched to MFN (full, $H = 64$) at approximately 94K parameters. GRU and LSTM hidden dimensions are adjusted to $H \approx 110$ to match. Ablations remove one mechanism at a time while keeping $H = 64$. All experiments are run for five random seeds and report mean $\pm$ standard deviation.

Model	Description	~Params
GRU (matched)	Single GRUCell, $H = 110$	~94K
LSTM (matched)	Single LSTMCell, $H = 95$	~94K
Mamba-tiny	Selective SSM, $d_{\text{model}} = 64$	~94K
xLSTM-small	Exponential-gated LSTM, $H = 90$	~94K
MFN (no fatigue)	$\phi = \mathbf{0}$ always; γ removed	~92K
MFN (no bidirectional)	$g_{\Lambda \rightarrow \Psi} \equiv \mathbf{1}$; $W_{\Lambda \rightarrow \Psi}$ removed	~90K
MFN (no gates)	$g_{\Psi \rightarrow \Lambda} = g_{\Lambda \rightarrow \Psi} \equiv \mathbf{1}$	~88K
MFN (shared decay)	Single MLP for both streams	~93K
MFN (full)	All mechanisms active, $H = 64$	~94K

Table 3: Models and ablations. 5 random seeds; mean $\pm$ std reported.

8.2 Task 1 — Multi-Delay Copy Task

Input $\mathbf{x}_{1:T} \sim \mathcal{N}(\mathbf{0}, I_d)$, $d = 10$, $T = 100$. The model must output $\mathbf{y}_t = \mathbf{x}_{t-k}$ for delays $k \in \{5, 10, 20\}$, and $\mathbf{y}_t = \mathbf{0}$ for $t \leq k$. Loss: MSE over $t > k$. Batch size 32, Adam $lr = 10^{-3}$, 500 epochs.

Delay $k = 5$ sits within the expressive stream's natural horizon; $k = 10$ and $k = 20$ require the conceptual stream. This makes the task directly discriminative between single- and two-stream architectures. The no-fatigue and no-bidirectional ablations are expected to degrade at $k \geq 10$, isolating which mechanism contributes to long-range retention.

8.3 Task 2 — Character-Level Language Modeling (PTB)

Standard Penn Treebank train/validation/test split. Metric: bits per character (BPC) on the test set. Sequence length 256, batch 64, Adam $lr = 10^{-3}$ with cosine annealing. Single seed (standard protocol). This task tests whether the cognitive architecture yields practical language modeling gains over standard recurrent baselines under an identical parameter budget.

8.4 Task 3 — Dual-Context Tracking

A novel diagnostic task designed to directly test *simultaneous maintenance of two independent signal sources*. At each step t , the input is a concatenation of two independent signals $[\mathbf{a}_t; \mathbf{b}_t]$, $\mathbf{a}_t, \mathbf{b}_t \sim \mathcal{N}(\mathbf{0}, I_5)$. The model must simultaneously output $\mathbf{y}_{\mathbf{a},t} = \mathbf{a}_{t-5}$ and $\mathbf{y}_{\mathbf{b},t} = \mathbf{b}_{t-10}$. Models with a single hidden state face a representational bottleneck: the two memory traces interfere. Models with two separated streams can in principle assign each trace to one stream. $d = 10$, $T = 100$, 5 seeds.

8.5 Fatigue Profile Analysis

After training full MFN on Task 1 ($k = 10$), we run inference on 100 held-out sequences and log $\phi_{\Lambda,t}$ and $\phi_{\Psi,t}$ at each timestep. We report:

1. Mean fatigue level by stream across the sequence.
2. Fraction of neurons with $\phi_{s,t,i} > 0.5$ ("saturated") at each step.
3. Pearson correlation between $\phi_{\Lambda,t}$ and $\phi_{\Psi,t}$ — high correlation would indicate the streams fatigue in lockstep, undermining the independence claim. A healthy architecture should show decorrelated profiles and temporal variation in the saturation fraction.

9. LIMITATIONS AND FUTURE WORK

Quadratic hidden cost.

GRU cells and feedback projections scale as $O(H^2)$ per step, matching standard GRUs but lagging behind linear SSMs such as Mamba and Griffin. Structured factorisation (e.g., Monarch [\[18\]](#)) of the GRU weight matrices would reduce this without changing the architectural semantics.

Fixed number of timescales.

The dual-stream design is manually specified. A natural extension would learn a mixture over N timescale streams via soft attention over a bank of GRU cells, analogous to mixture-of-experts architectures.

Scalar β for slow-stream decay.

The exponent $\beta = 0.2$ is a global hyperparameter. Learning β per-neuron as a trained scalar would allow heterogeneous forgetting rates within the conceptual stream, better reflecting the biological reality that different neuron populations have different memory time constants.

No auxiliary cognitive loss.

MFN is trained solely on the task loss. Adding an auxiliary loss that rewards the conceptual stream for predicting future inputs — connecting MFN to predictive coding theory [\[11\]](#) — would give the inner stream an explicit anticipatory function, potentially improving long-range generalisation even when the task loss alone does

not require it.

Input-independent fatigue recovery.

While γ_s is learned, it is input-independent. A dynamic recovery rate — one that accelerates when the input changes rapidly and slows during stable inputs — would better model activity-dependent recovery observed in biological synapses, implementable as a second MLP conditioned on the current input and previous fatigue level.

Evaluation scope.

The current experimental suite covers sequential signals and character-level text. Audio, video frame prediction, and multi-speaker dialogue — where the inner and expressive streams have natural cognitive referents — remain important future benchmarks.

10. CONCLUSION

We introduced MyelinFatigueNet, a recurrent architecture that decomposes hidden dynamics into an expressive stream $\mathbf{h}_\Lambda$ and a conceptual stream $\mathbf{h}_\Psi$, coupled bidirectionally by two independent learned confidence gates and regulated by per-stream neural fatigue accumulators. Fatigue is modeled as an exponentially decaying activation trace per neuron, with learned per-neuron recovery rates; its effect is to prevent representational fixation and force diversity of activation across time — a functional property absent from all prior sequence models. Bidirectional coupling allows background reasoning to guide surface expression while expression simultaneously feeds back into conceptual updating.

We derived the full parameter budget as $22H^2 + (3d + 2d_{\text{out}} + 15)H + d_{\text{out}}$, with the dominant $12H^2$ contribution from the two GRU cells. Three complementary evaluation protocols — multi-delay copy, character-level language modeling, and dual-context tracking — provide a discriminative experimental protocol isolating the contribution of each mechanism through ablation. We release the full reference

implementation and hope MFN serves as a useful building block for streaming, real-time, and cognitively-motivated sequence modeling systems.

REFERENCES

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
2. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
3. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP 2014*.
4. Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166.
5. Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*.
6. Gu, A., Goel, K., & Ré, C. (2021). Efficiently modeling long sequences with structured state spaces. *ICLR 2022*.
7. Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33.
8. Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. *Proceedings of ACL 2019*, 2978–2988.
9. Chung, J., Ahn, S., & Bengio, Y. (2017). Hierarchical multiscale recurrent neural networks. *ICLR 2017*. *arXiv:1609.01704*.
10. Koutník, J., Greff, K., Gomez, F., & Schmidhuber, J. (2014). A clockwork RNN. *Proceedings of ICML 2014*, 1863–1871.
11. Rao, R. P., & Ballard, D. H. (1999). Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.
12. Lotter, W., Kreiman, G., & Cox, D. (2016). Deep predictive coding networks for video prediction and unsupervised learning. *arXiv:1605.08104*.
13. Zucker, R. S., & Regehr, W. G. (2002). Short-term synaptic plasticity. *Annual Review of Physiology*, 64, 355–405.
14. Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). xLSTM: Extended long short-term memory. *arXiv:2405.04517*.

15. De, S., Smith, S. L., Fernando, A., Botev, A., Cristian-Muraru, G., Gu, A., Haroun, R., Berrada, L., Chen, Y., Srinivasan, S., et al. (2024). Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv:2402.19427*.
16. Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., et al. (2023). RWKV: Reinventing RNNs for the transformer era. *EMNLP 2023*.
17. Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., & Wei, F. (2023). Retentive network: A successor to transformer for large language models. *arXiv:2307.08621*.
18. Dao, T., Chen, B., Sohoni, N. S., Desai, A., Poli, M., Grogan, J., Liu, A., Rao, A., Rudra, A., & Ré, C. (2022). Monarch: Expressive structured matrices for efficient and accurate training. *ICML 2022*.